

TUTORIAL SESSIONS



5210COMP:

Software Engineering for Games

Module Leader: Dr Chris Carter

Tutorial 08: UE5 Tutorial #01 Controlling the Player and Targets in the UE5 Template

Tutorial Overview

In today's tutorial we will be looking at the individual aspect of the coursework assignment, which tasks you with refactoring an existing UE5 solution written in Blueprints.

The tutorial works directly with the coursework template – any work that you complete in these UE5 tutorials, you are free to integrate into your own solution to the coursework – it is advised that you have separate copies of the project rather than trying to do the tutorial directly on your coursework attempt.

We will also be working with the Editor's use of the gameplay framework to switch between our two different characters – these two separate characters need unifying into a single character that can switch between the different weapons.

In **Task 1**, we will be setting up the application and looking at some of the key structures of the template.

In **Task 2**, we will see how we can use the editor to place multiple targets and control them.

In **Task 3**, we will be creating a new Actor using Blueprints to act as a manager over our targets and cover some of the key processes involved in handling/filtering multiple actors.

In **Task 4**, we will deal with some basic movements related to the Targets.

THIS IS A BIG TUTORIAL!

IT DEMONSTRATES SEVERAL CONCEPTS RELEVANT TO THE COURSEWORK – I WOULD ADVISE DOING IT A TASK AT A TIME AND TAKING A BREAK BETWEEN EACH TASK – THIS WILL TAKE LONGER THAN A TYPICAL TUTORIAL TO COMPLETE – PARTICULARLY IF YOU ARE NOT USED TO BLUEPRINTS YET.

Task 1: Game Modes and Characters and the UE5 Gameplay Framework

In this task we are going to make sure we understand the Basics of working with the application template.

Step 1: Obtain a copy of the UE5 coursework template

This step assumes you have downloaded and set up Unreal Engine 5.2 on your development machine already (or are working from the Lab machines on campus, where 5.2 is pre-installed). If you need help and support with that, consult the **Support – Working from Home** section on Canvas.

IDEs, Development Tools and Engines	✓	:
7-Zip Archive Manipulation Tool	✓	:
Info: Obtaining Visual Studio 2017/19 from Azure Dev Tools for Teaching	✓	:
Azure Dev Tools for Teaching - LJMU Sign-In Portal	✓	:
Info: Obtaining Unreal Engine 4	✓	:
Page		
Info: Unreal Engine: Epic Launcher Login Credentials	✓	:

If UE5 is already set up, then download the coursework template – this can be found under the **Assignment 1 – UE5 Refactoring Task** section.

Download and extract the Zip file (*it is suggested you use 7-Zip for this*) to an appropriate location on your machine (try not to make your file path too long, so a folder like C:\SEG-UE5\ on your own machine would be appropriate – on campus, use the Local folder we created in the SFML tutorials).

You should end up with a folder called **SEG_FiringRange502**.

Name	Type	Size
Config	File folder	
Content	File folder	
DerivedDataCache	File folder	
SEG_OO_FiringRange.uproject	Unreal Engine Proj...	1 KB

When completing the tutorials, it is not recommended you do the work directly in your coursework version, so I suggest renaming the SEG_OO_FiringRange folder to **SEG_Tutorial08**.

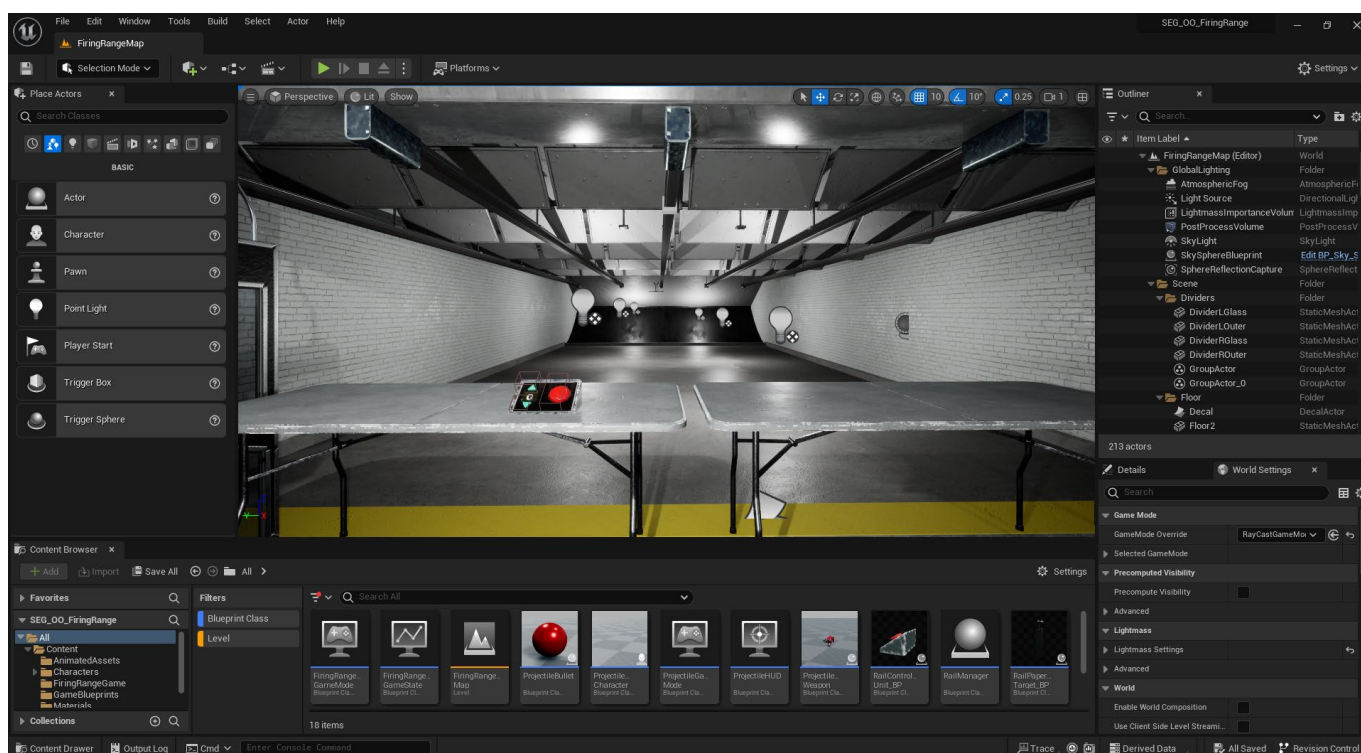
As long as you don't alter the names/or structure of the files inside that folder, you are free to rename the outer folders with UE5 projects.

Open the project by double-clicking on the .uproject file – this is a UE5 project file. This stores the modules we will use of the engine, the version of the engine and any plugins – it tells the engine how to compile our project.

Name	Type	Size
Config	File folder	
Content	File folder	
DerivedDataCache	File folder	
Intermediate	File folder	
Saved	File folder	
SEG_OO_FiringRange.uproject	Unreal Engine Proj...	1 KB

Whilst the Editor is opening, it will compile the project for development – this will procedurally generate new folders for things like shaders, temporary files etc., as shown above.

The project should be open with our Firing Range map.

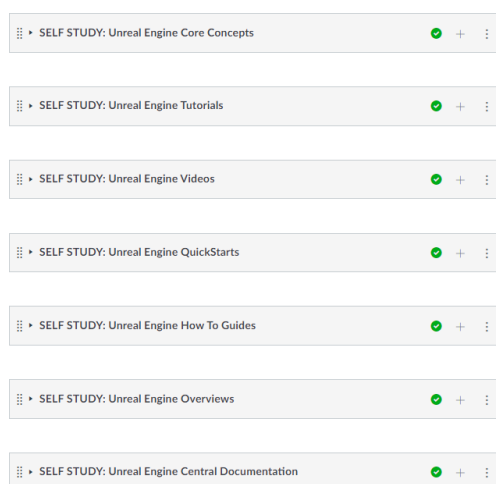


Step 2: Getting up to Speed with UE5 Blueprints

In this module, we will be focusing on effective OO Programming inside UE5, applying the theory we have been learning in the lectures and SFML tutorials, to a AAA game engine, and in a language other than C++ (to prove the OO paradigm is not tied to a language, and also how understanding OO will help you learn new programming languages that will benefit your career as a games developer).

Therefore, similarly as we have not re-covered the basics of C++, it is expected you familiarize yourselves with the basics of UE5 and the Blueprint scripting system.

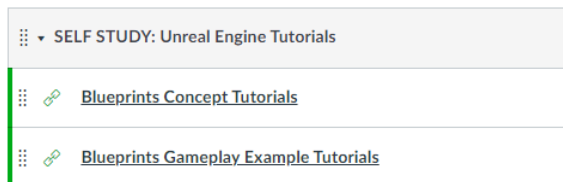
To help with that, there is lots of content provided for you – these are at the bottom of the 5210COMP modules section:



You don't need to consume all this material – *I am not going to prescribe an approach to you for this* – I have simply provided you with content in a variety of formats, from videos, to documentation, to QuickStart step-by-step tutorials, to overview tutorials.

It is up to you to work out which of these will be best suited to you and most appropriate to review.

Having said that, I would recommend at least working through the two basic tutorial sections in the Unreal Engine Tutorials area, as these cover some of the key tasks that are relevant for this assignment.



I also highly recommend that you create your own Epic Games account and then log into the UE5 Dev Community.

[Learning | Epic Developer Community \(epicgames.com\)](https://www.epicgames.com/learn)

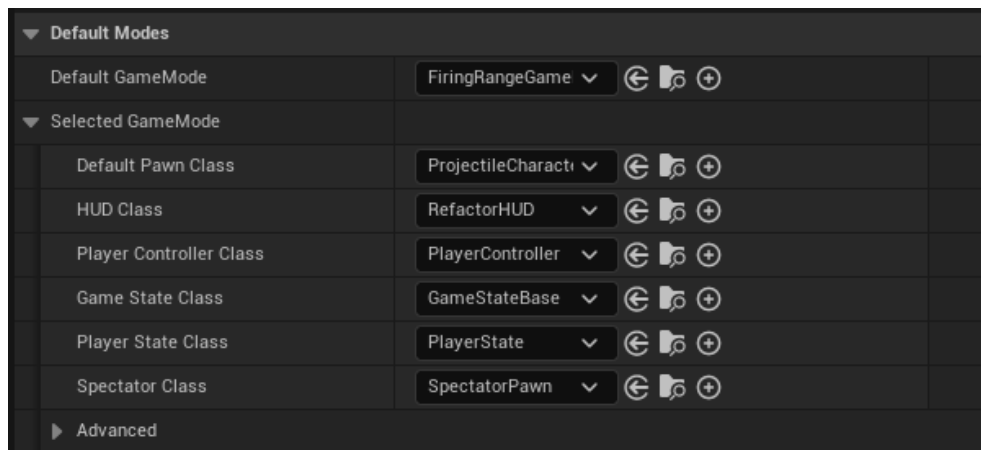
You don't need to do these before completing today's tutorial, but I urge you to complete some of them ASAP in order to help you get used to working in UE5 Blueprints and the Gameplay Framework, this will help both with this coursework and future Unreal based courseworks.

Step 3: Working effectively with the Game Mode system.

For each game to run, we need two aspects defined – the **Map** to open and the **game mode** which controls our execution.

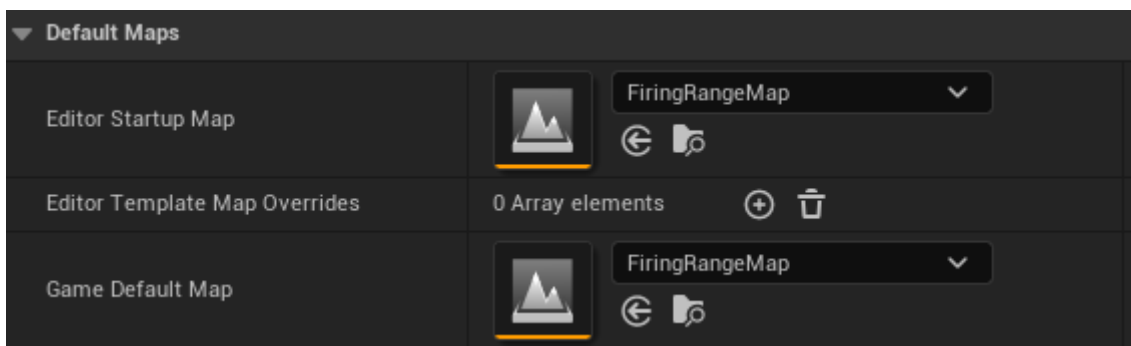
The default game mode is stored in project settings and is customizable from these as well.

1. On the Menu Bar above the viewport, select the Settings Dropdown on the far right, and pick 'Project Settings'. Go to 'Maps and Modes':



Our default game mode is **FiringRangeGameMode**.

This has been configured with the other Blueprint classes to be used for various aspects of the game. Below that, you can see the default setting for Map:



In both cases, you can see the Map is **FiringRangeMap** in both cases. The first option is when we open the project in the Editor and the second option is run when we put the game in standalone mode (without the editor) – in most cases, these will be the same.

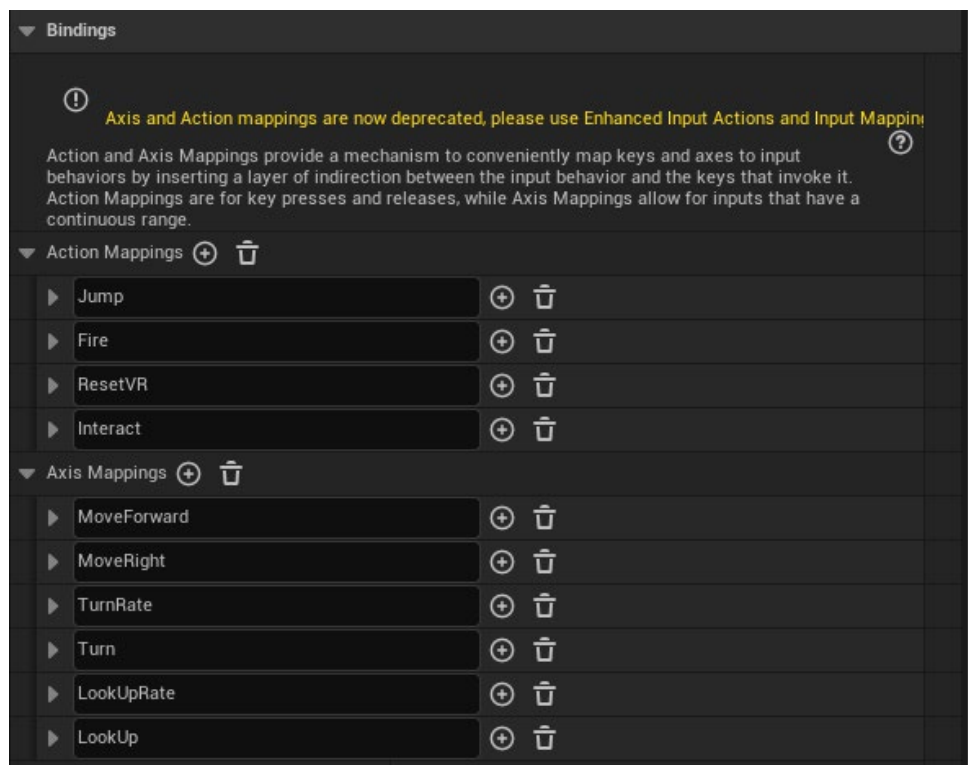
The other project setting that is most relevant is our **Input Control Scheme**.

In this application, we are using an abstraction between the keyboard/mouse/gamepad and the inputs of the game, just as we have taught you in this module. This is using the UE4 technique for input and not the new Enhanced Input system, **which we will touch upon in later tutorials**.

UE5 handles this abstraction by storing Input actions and Input axes for physical input devices and associating them with a text identifier.

2. You can see these inputs by going to the Input category on the left-hand side.

If you want to handle more types of Input, this is where you will do so by adding your own custom actions.



In the template, there are four actions, and six axis values.

There are two types of turn and look, because they work slightly differently for Mouse vs Gamepad; therefore, we basically have the following control scheme.

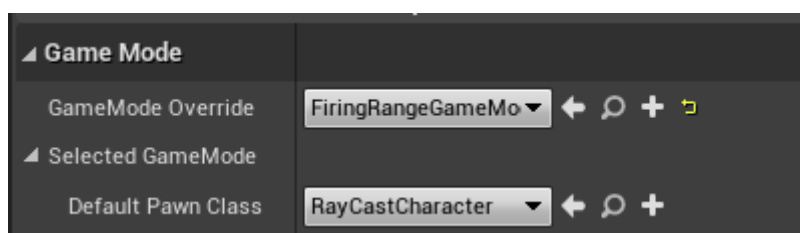
Action Name	Mouse / Keyboard	Gamepad (e.g. PS4 Pad)
Move Forwards/Back	W/S or Up Arrow/Down Arrow	Left Thumb stick
Move Left/Right	A/D or Left Arrow/Right Arrow	Left Thumb stick
Look Around	Mouse Movement	Right Thumb stick
Jump	Space	Cross (X)
Fire Weapon	Left Mouse Button	Right Trigger (R2)
Interact.	Enter	Square (□)

We can override individual settings of the game mode as we are developing, outside of the Project Settings (as long as it is a blueprint based GameMode – C++ game modes can only be modified in code).

3. Close the Project Settings and Start the game.
4. Fire the Weapon using the input controls above and you should see that we have a Projectile-based Character by default.

Next, Open the World Settings Tab (below the World Outliner, right-side tab, or Window Menu Bar -> World Settings is checked) and you should see a GameMode override.

5. Set this to FiringRangeGameMode (our default game mode) and then find the **Selected GameMode** section – expand the node and change the **Default Pawn** class to **RayCastCharacter**.



Play the game and you should see that the Fire is behaving differently.

6. Try changing the Pawn to one of the in-built Pawns such as ArchVisCharacter.

You should be able still control this character **because the input actions used in that class (turn, move etc.) are the same names across all of these characters.**

As well as changing individual classes such as Pawn class, we can control the whole collection of classes by changing the game mode itself. Game Mode classes define the concrete classes we are using for each key class of the UE5 Gameplay Framework.

7. Read through the description of each type of class here: [Gameplay Framework Quick Reference in Unreal Engine | Unreal Engine 5.2 Documentation](#)

The Game Mode's job is to specify and create instances of these classes and then Spawn them into the World.

8. Change the Default Pawn class back to **ProjectileCharacter** and then reset the game mode back to default by clicking the left arching arrow which is to the right of the + sign on GameMode Override – this will return to defaults (Override is None).
9. Play the game and Press F8.

This ejects you from the controlled character – this mode is good when you want to test out changes to the game whilst it is running. You should have mouse control back – go to the World Outliner and scroll through the objects – at the bottom you should see some Yellow highlighted objects.

Item Label ▲	Type
▼ FiringRangeMap (Play In Editor)	World
▶ GlobalLighting	Folder
▶ Scene	Folder
FiringRangeGameMode0	Edit FiringRangeGameMode
GameNetworkManager0	GameNetworkManager
GameSession0	GameSession
GameStateBase0	GameStateBase
PaperTarget1	Edit RailPaperTarget_BP
PaperTargetControlUnit1	Edit RailControlUnit_BP
PaperTargetFollower1	Edit RailTargetFollower_BP
ParticleEventManager0	ParticleEventManager
PlayerCameraManager0	PlayerCameraManager
PlayerController0	PlayerController
PlayerStart	PlayerStart
PlayerState0	PlayerState
RailManager	Edit RailManager
RayCastCharacter0	Edit RayCastCharacter
RayCastWeapon0	Edit RayCastWeapon
RefactorHUD0	Edit RefactorHUD

This means they have been dynamically spawned into the game whilst it is running. All the spawned classes in this case are the ones controlled by the Game Mode. The blue hyperlinks can be used to open the Blueprints (if it says Edit) or the C++ class source code (if it says Open) to inspect them. Don't do this yet.

10. Next, change the GameMode Override to **RayCastGameMode**. You should see that this changes not just the Pawn, but also the HUD class as well. Test that this changes the character.
11. Finally, change it to **ProjectileGameMode**. This has the same settings as the FiringRangeGameMode.

There are three game modes in the template because I want you to be able to test out your gameplay with both the **ProjectileCharacter** and **RayCastCharacter**, but the **FiringRangeGameMode** (or your own custom Game Mode) should be used for your actual game.

In aim of the coursework is for you to replace the default character with your own implementation of a character (using the provided RefactorCharacter as the basis) that can switch between the two types of weapons.

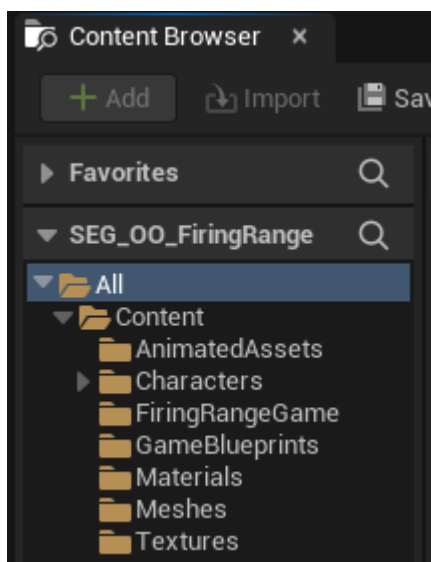
The link between the game mode and the other classes is the key element of the gameplay framework – Unreal Engine has always worked like this because of the types of game Epic make (e.g., *Unreal Tournament* had *Capture the Flag* and *Deathmatch* Game Modes; *Gears of War* had *Co-op* and *Horde* mode; *Fortnite* has *Save the World* and *Battle Royale* etc.).

This is the best way of working with the engine – the Maps are that – they are levels representing our game world – don't use them to place game logic – this is the job of these classes here.

Step 4: Customizing the Game Mode

To demonstrate how we can use the game mode to control our game logic and create new classes for the framework defaults, we will perform a couple of simple actions.

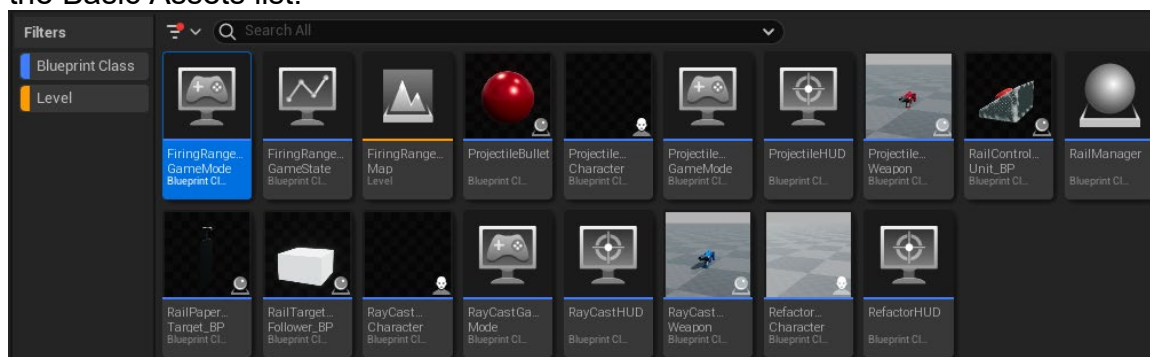
1. Start by working with the Content Browser/ContentDrawers. Make sure the Icon to the Left of Filters is selected so you can see the folders of the project.



I have tried to structure the content to make it easy to work with – most the folders are self-explanatory – click in each folder to have a look at the type of content that is present in each. You are free to change/modify/replace the content in each of the folders if you fulfil the game specification.

However, most of our work will be across the Map and Blueprints, so we want to easily work with them. The best way of doing this is via **Filters**.

2. Make sure you have selected the Content folder itself (should be highlighted in Yellow-Orange) and then from the Filters dropdown select 'Level' and 'Blueprint Class' from the Basic Assets list:



This set of files sums up the classes that comprise this game – we have our Three Game Modes, two user interfaces (HUD classes), our three different character types and then the rest of the classes represent the bullets and the Target System.

3. Open **FiringRangeGameMode**. This is a data-blueprint by default, which means it is just changing the data state of the class it inherits from (**GameModeBase**) – we can use this mode to easily change data values.

NOTE: This is a data only blueprint, so only the default values are shown. It does not have any script or variables. If you want to add some, [Open Full Blueprint Editor](#)

However, we want to add some behaviours, so click the 'Open Full Blueprint Editor' link to see the whole class.

Hopefully, you are now familiar with the Blueprint Editor to some degree from 4225COMP – otherwise, you should really look at the basic UE5 tutorials on the Editor.

The **My Blueprint** Section is what we are interested in – this is equivalent of a C++ Header file – it defines the Data and declaration of functions etc.

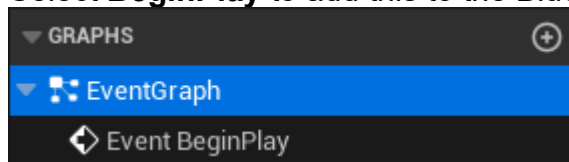
We are going to override one of the many functions of game mode.

4. Click the Add button and then -> Override Functions -> Begin Play.

The override functions section lists all the functions of this class that we can override/extend via inheritance.

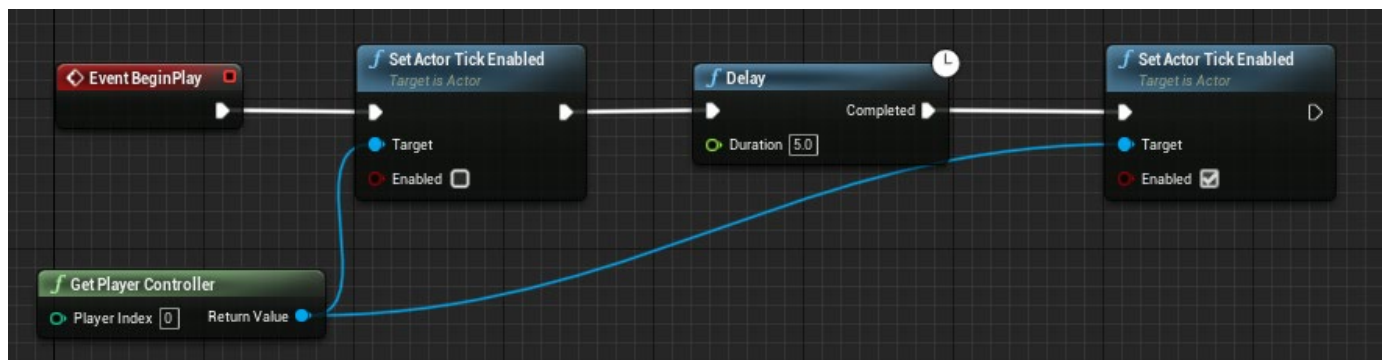
It also tells you what parent class they belong to – since GameMode inherits from Actor, we see both Game Mode functions AND Actor functions – the deeper the inheritance, the more functions you will see.

Select **BeginPlay** to add this to the Blueprints Event Graph:



5. Try and complete this with the following Blueprint nodes in the Event Graph. The more you practice, the quicker you will get at adding nodes.

Some are easier to find by dragging from the pin; others are easier if you right-click in a blank space and type in the start of the node name.



This is very simple; we are just going to get the player controller of our player and then turn off ticking (which means it won't receive updates – like the update function in our GameState calls Player update) – we then wait for 5 seconds and then re-enable tick.

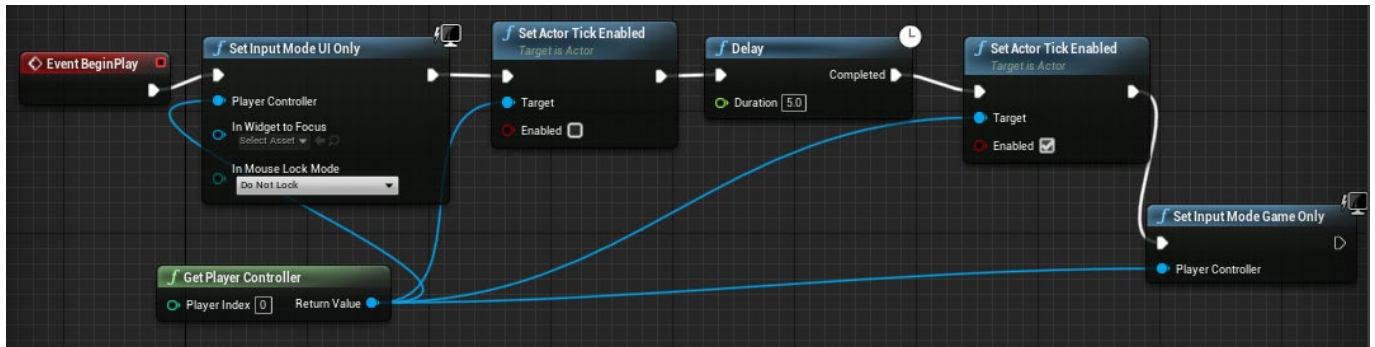
6. Compile the blueprint and then Save it using the menu bar above the event graph and run the game.

You should find that the game's character doesn't respond for five seconds.

However, if you have pressed the Fire button a few times, you might find that bullets spawn straight away – this is because although the actor is not ticking, it has received input events and

has queued up a set of actions to be performed. Therefore, we also need to stop the player from responding to input as well.

7. Change the **BeginPlay** logic to the following:



Compile and save; test it out by running the game.

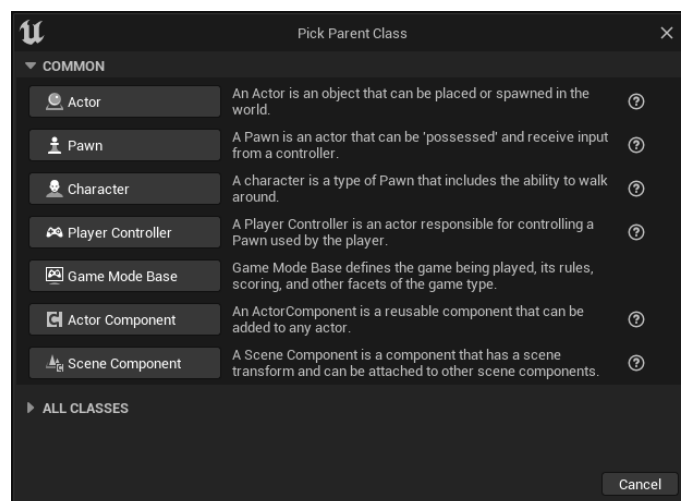
By switching Input Mode to UI only, we enter a mode that is designed for mouse-driven menu systems – when we enable the actor, we return to game input which means our input controls work again. This time, no actions should be queued against the player whilst we are not ticking.

We have demonstrated that we can control the game flow easily via the Game Mode, next, we will alter the default gameplay classes.

Step 5: Adding State to the Game

Unreal has two classes designed to hold state (game data), which are **PlayerState** and **GameStateBase** – the difference should hopefully be obvious – one is focused on player specific state, whilst the other is state that is global to the game itself. We'll create custom instances of both.

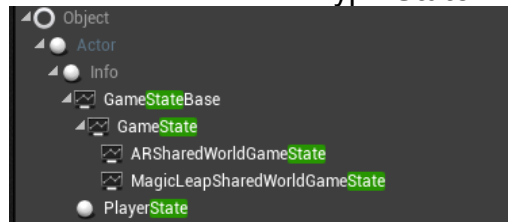
1. In the Content browser, navigate to the *FiringRangeGame* folder and then right click inside the content window next to the other file icons and select **Create Basic Asset - > Blueprint Class**. The pick parent class window should appear:



In most cases, you will want to inherit from one of these seven classes – they are indeed the most used.

This time however, we are going to use the two state-related classes mentioned above.

- Expand the All Classes view at the bottom and type **State** in the search bar.



This view is very handy as it shows our inheritance hierarchy – both classes are sub-classes of Actor, but not direct descendants – instead, they are grand-children of Actor and extend directly from one of Actor's child classes – Info.

Info is a special type of Actor designed not to be positioned in the 3D Game world.

We use this when we want to create classes which manage other classes – it still gets updated by the engine as part of our game loop but is not visible to the player.

I really recommend making use of Info to create Actors which deal just with the logic of our game (e.g., *Quest Managers*, *Skill Trees*, *RTS Unit Managers*, *AI Directors* etc. are all great candidates for Info).

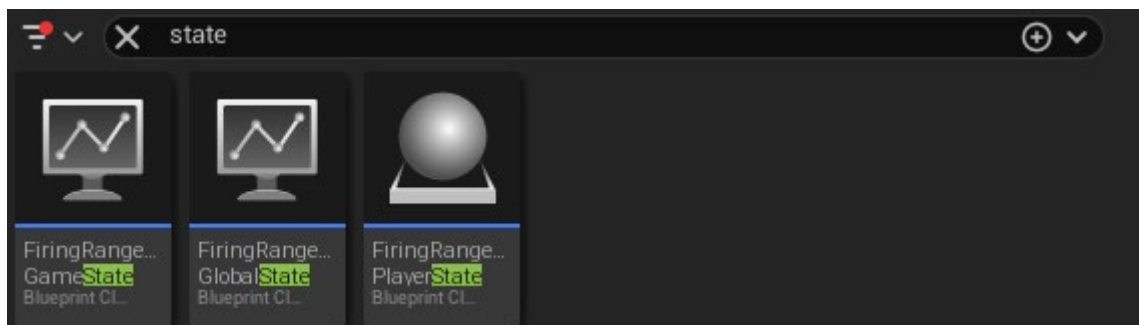
- Select **GameStateBase** (*not GameState! this will break the code*) and then choose Select to create the class.

Name it **FiringRangeGlobalState**.

Once created, go to File -> Save All to make sure everything is saved – you should do this regularly, as UE5 often doesn't save files completely, but rather autosaves them temporarily – this catches a lot of people out.

I will only be marking you on what is saved correctly – when we re-download the projects for marking, these temporary files are destroyed.

- Repeat the steps to create a new class which extends from PlayerState, called **FiringRangePlayerState**

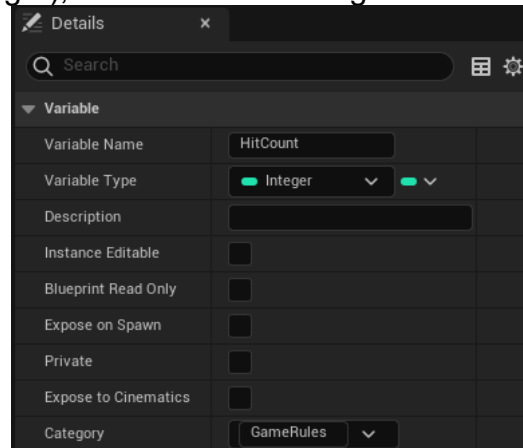


Open the Firing Range Global State in the Blueprint Editor.

Switch to the Full Blueprint Editor.

We'll add some custom data to this blueprint – these are class members, just like C++.

5. Hover over the variables option and click + Variable – call this value **HitCount** and in the Details view (Top-Right), set it to the following.

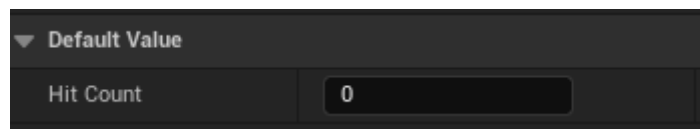


To create the Category GameRules, don't use the dropdown, just type into the TextBox.

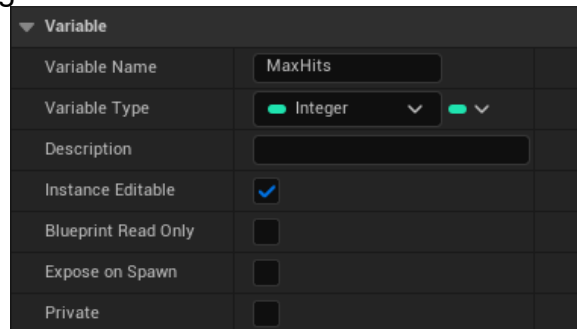


Notice how spaces are added where the Capitals are used.

6. Hit the Compile Button – the Default Value view should be visible now – leave this at 0



7. Next, add the following field:



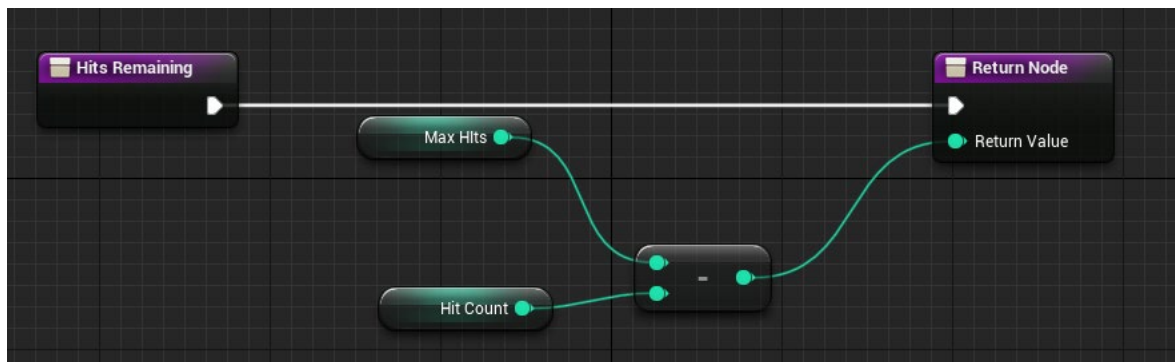
Making it Instance Editable means it can be changed in the Editor Details pane (so *game designers can customize it*).

Once added, in the Variable view, drag the variable into the GameRules category.

8. Add a new Function using the **+Function** button and rename it HitsRemaining. Complete this as follows.

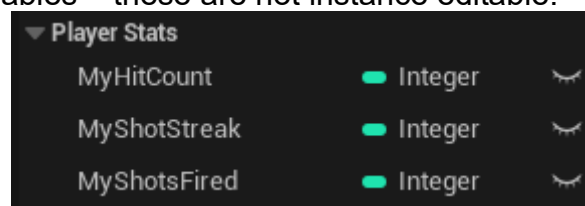
TIP: You can hold Ctrl whilst dragging the variables into the Event Graph to access them as getters.

Return nodes can be added by typing Add Return into the right-click search and then dragging the pin you want to return onto the node.



- By yourselves, in our new **FiringRangePlayerState** class (don't put this data into the global game state), that we created in our earlier step (revisit the last few pages if you do not have a FiringRangePlayerState blueprint)

Add the following variables – these are not instance editable.



These are all integers, try and add them to the same Category as shown above.

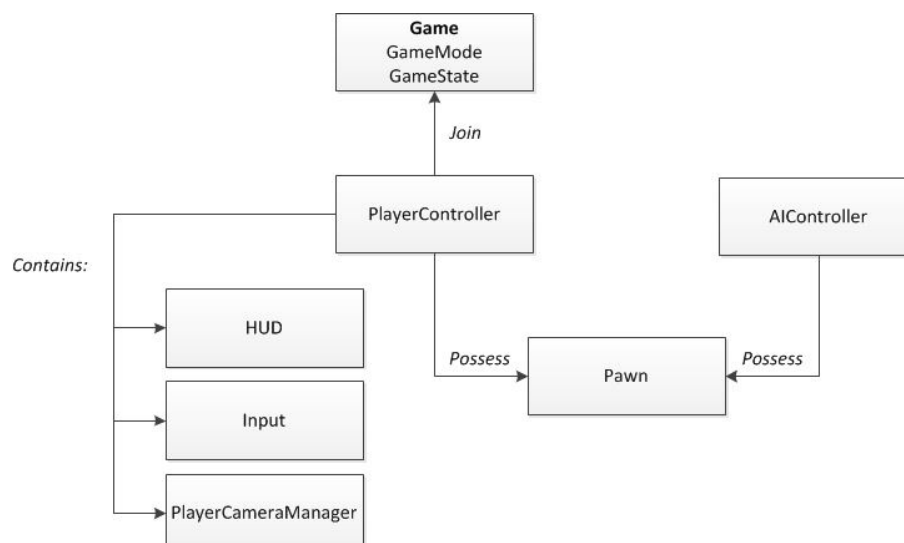
Where we store this data becomes more important in Multiplayer/Networked games.

For local single player games, we could store both the global game state AND player state in the game mode.

It is worth getting used to the correct way of using the engine, as early as possible, even if it is slightly more work to do so.

Step 6: Setting the State on the Game Mode and Accessing it

Now we have the states in place, we need to associate them with the GameMode – the relationships between the different classes are as follows:



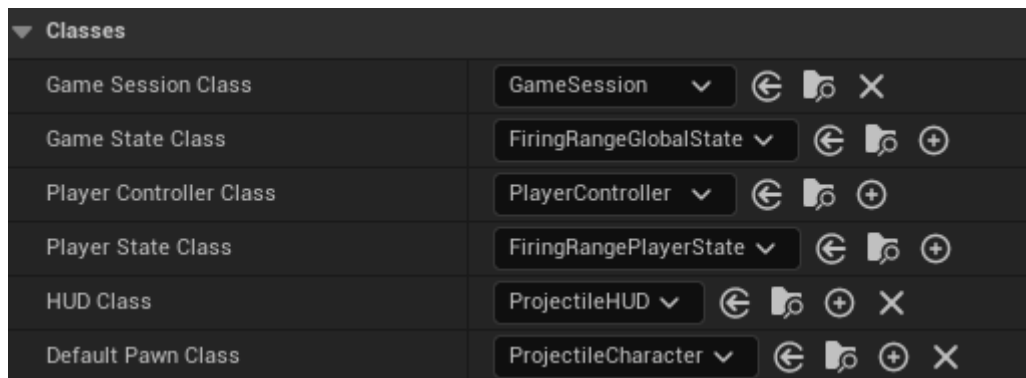
PlayerState is also associated with the PlayerController.

That means that regardless of how many pawns we are controlling or if we switch pawns (e.g., *change from a player to a vehicle*), we will still have this one copy of the state.

Player State IS lost when we change levels (but we can copy it across easily, before transitioning between levels).

For information that out-lives a level, we have another class called GamelInstance – we will get to that in another tutorial.

1. Locate **FiringRangeGameMode**, open it in the Blueprints Editor and select the 'Class Defaults' option from the menu bar – you should have the details view on the right-hand side.
2. Under 'Classes' set the PlayerState and Game State to each of our newly created Blueprints.

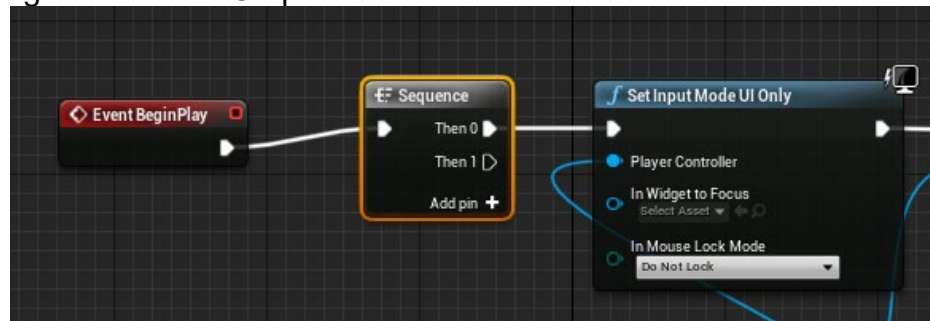


Notice that Unreal filters the dropdown to only classes which have the correct inheritance – this makes life much easier.

This change also means our new classes will be created when we start the game.

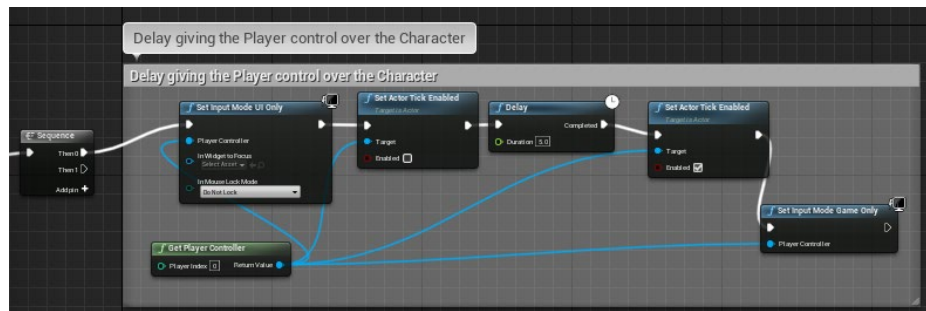
Next, we need to be able to access them directly from our Blueprints script.

1. Find our **BeginPlay** event and add a sequence node as the first node and then connect the first pin to our Set Input Mode node. You can add a sequence node easily by holding S and clicking in the Event Graph.



2. Next, select all the nodes to the right of our Then 0 pin (our previous code) – again, you can do this by dragging a rectangle around them whilst holding the Left Mouse Button. Right-Click and select 'Create Comment from Selection'.

- Click the Comment Text, so you can type your own Comment – I would put something like the following:

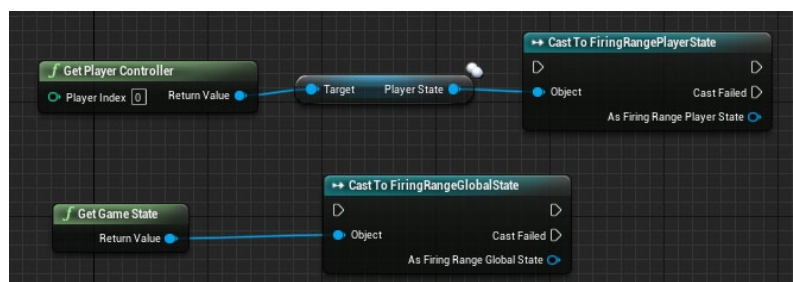


Both sequences and Comment blocks are a great tool to help us organize our Blueprints.

- When you move a comment, all nodes inside that comment are moved in tandem. You should consider Sequences as if they are **sub-routines** – they don't change the flow of execution, but it means that rather than adding more code to the right of our previous code, we can just create a second block of code elsewhere, and connect it to the **Then 1** pin, so that is executed after our original code.

It makes the Blueprint's cleaner and means if we want to alter our code later to change the order, we must move the sequence pin, not reorganize dozens of nodes.

- Add in the following nodes below our comment block:



That is why that Diagram above is important – we know where to locate our Classes – Game Mode has the Game State, so we just call Get Game State, and we access the Player State via the Player Controller.

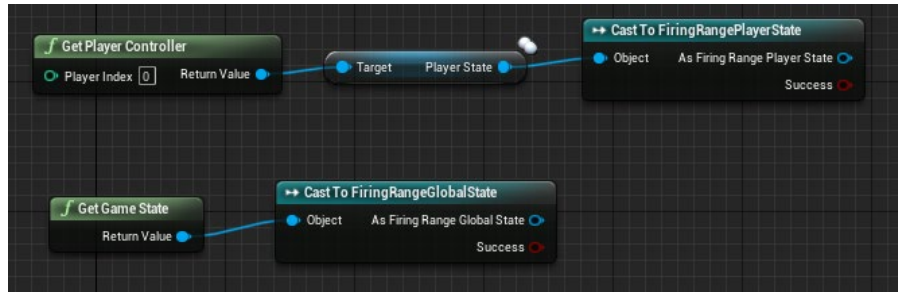
This returns the UE5 default parent classes, so we are forced to **downcast** to our specific class.

In this example, they are **Callable** nodes – we need to wire them up using an execution pin.

Our Getters are **Pure** Nodes, we don't need an execution pin for them to work.

Blueprints also support Casting as a Pure node, known as a **Pure Cast** – we should use Pure Casts when we are 100% that the cast will not fail (we know the sub-class). In this case, we know this is the case, because we have set them on the Game Mode.

- On each of the two Cast Nodes, right-click and select 'Convert to Pure Cast'. You will end up with the following:



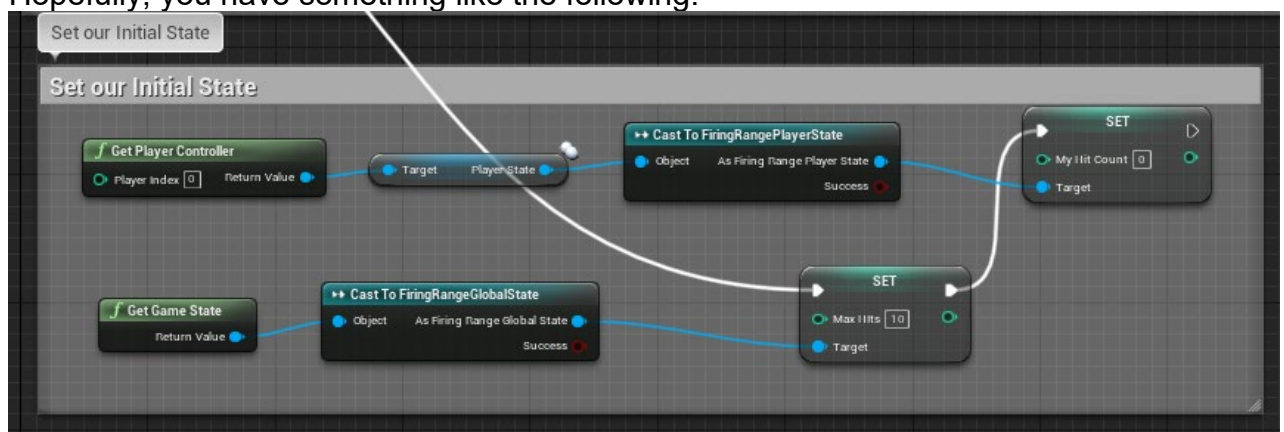
- Using the **As...** nodes to help you, set the Max Hits to 10 and the My Hit Count to 0.

HINT: To find these nodes, drag the **As** Pin and then type the Category that each variable belongs to into the search bar (look at the earlier screenshots to help you) - this is why Categories are useful – they help us organize and locate data.

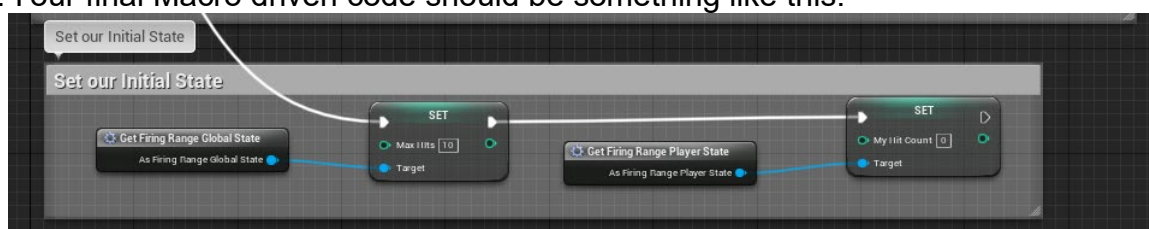
- Wire them up to the **Then 1** Pin so that Max Hits is set first, followed by the Hit Count.

Once this is done, select all the nodes we have just added and create a Comment Block.

Hopefully, you have something like the following:



- Finally, let's look at one final tool to help us organize our blueprints. Select the Three nodes that Cast to the FiringRangePlayerState and from the right-click menu, choose 'Collapse to Macro'.
- Rename the Macro to **GetFiringRangePlayerState**. Create a similar Macro for our **FiringRangeGlobalState** (e.g., **GetFiringRangeGlobalState**). Now, any time we want to get these State objects, we can re-use these Macros. They are also handy when we must do a lot of getting and setting of nodes, such as when we are creating a Mathematical equation. This should also help you organize your blueprint into a nice, linear set of nodes.
- Your final Macro driven code should be something like this:



OK, so let's briefly recap on what we have covered.

- We want to use the Game Mode to control our Application Flow and determine the major classes that make up our Player and our state.
- *We have the Game State and Player State classes for Data.*
- We know how to create Blueprints that derive from particular classes.
- *We know how to create variables in Blueprints as class members and both get and set them.*
- We know how to set the default value of those variables.
- *We know how to override functions in our classes.*
- We know how to create custom functions in a blueprint.
- *We know how to associate variables with Categories and why this is a good thing.*
- We understand that the gameplay framework links all these classes together – Game Mode and Player Controller are the lynchpins of this.
- *Begin Play is the function that is called when we are starting the game – this is called on all Actors that are part of a level.*
- We know that the Game Mode spawns these objects for us based on what we have chosen in the drop downs and also that we can change the Game Mode either in the Project Settings or override it for a particular Level.
- *We can create comment blocks, sequences, casting nodes (both callable and pure) and Delay nodes to help both organize and control the flow of a blueprint event or function.*

That is a lot of core functionality of Blueprints Development covered in a short amount of time! – whilst we have not actually added much to the game, each of these concepts will really help in learning to build blueprints and working with UE5 effectively – the next steps are really to learn what nodes of the engine to use in what situations.

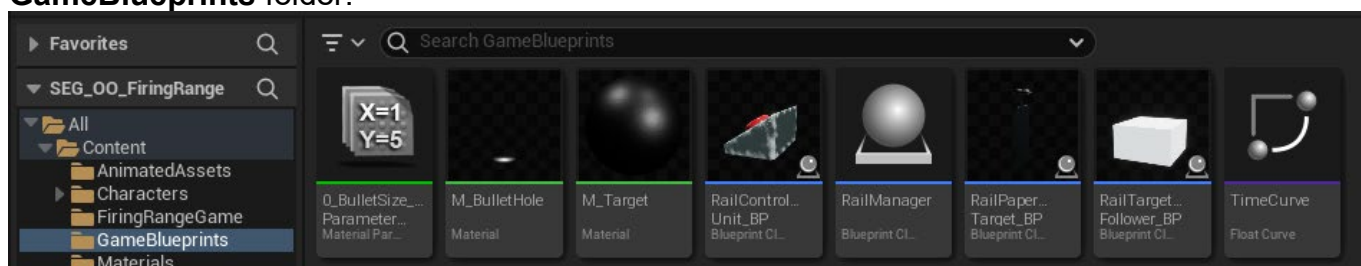
Task 2: Placing New Gameplay Elements into our Game World – working with the Targets

This second task is much shorter and simpler than the first, but we will also look at how the template works as part of the process – we are just going to use our existing Blueprints to add new Targets into our Game World.

Step 1: Locate the Files which make up the Target Mechanic

Let's start by finding the relevant files:

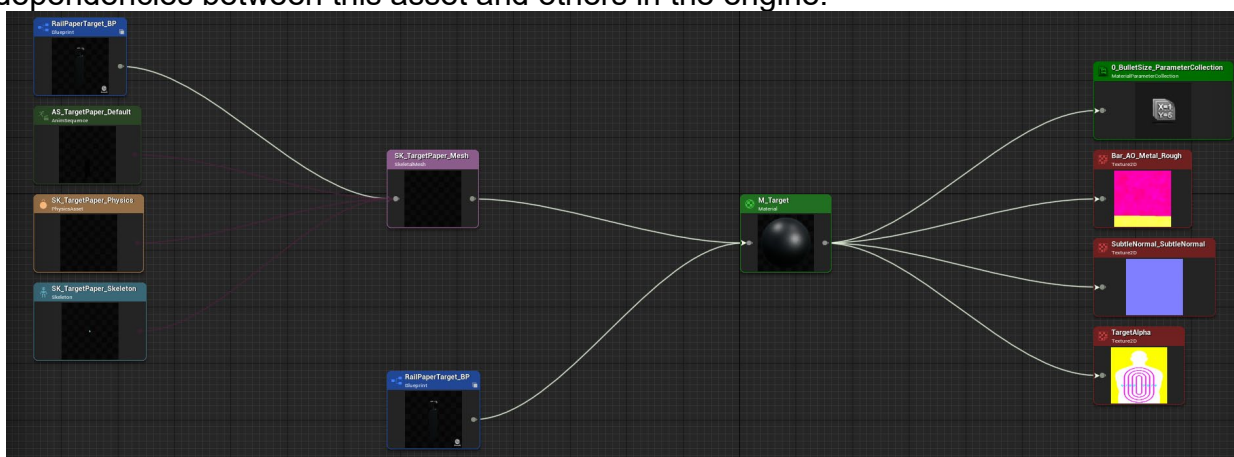
1. In Content Browser, select 'Reset Filters' from the Filters dropdown and then go into the **GameBlueprints** folder.



We have eight assets which make up the Target system – Four of these are Blueprint, we also have two materials, a set of Material parameters and a Timeline.

The technical details of graphical engine materials is outside the focus of the tutorials (*and the module as a whole*), but it is important to understand where they are used.

2. Select 'M_Target' and Right Click and Select 'Reference Viewer'. This shows the dependencies between this asset and others in the engine.

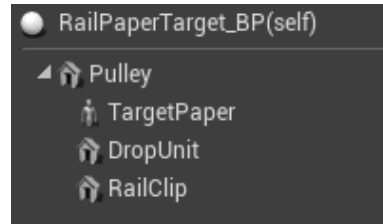


We can see the Material is depending on three textures and our parameter collection, but also, two assets are dependent on it – which are the Skeletal Mesh of the Target and the **RailPaperTarget_BP**. This is because this Material represents the Target Material with the Body Target. That is overlaid onto the Target Mesh.

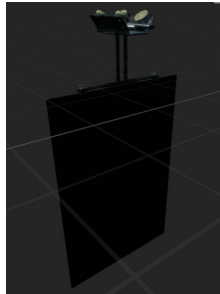
Step 2: Understanding the Target

1. Right-Click and duplicate the **RailPaperTarget_BP**.

2. **Open the duplicate.** Let's have a look at the structure of this Blueprint.



Select the 'Viewport' Tab.



The **Pulley** is our Root Component, with the Skeletal Target Mesh **TargetPaper** representing the Target itself, a Static Mesh called **DropUnit** and the **RailClip** attached to this Pulley. Therefore, when we move the Actor, or the Pulley, everything else moves relative to that.

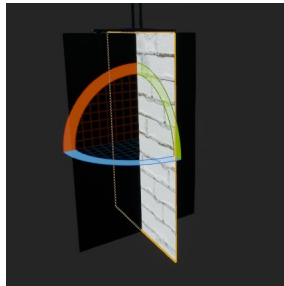
The RailClip is the simplest, it is a Static Mesh which is aligned to it is connected to the Pulley, and then the Target Paper is positioned at the bottom of it, to look like they are connected.

3. One change you might consider is to re-organize the Components, so that the **TargetPaper** and **DropUnit** are children of the RailClip – you can do this by dragging them onto the RailClip:



You must think about this Hierarchy when planning gameplay. This will affect collisions etc., which is why we are applying it to the duplicate.

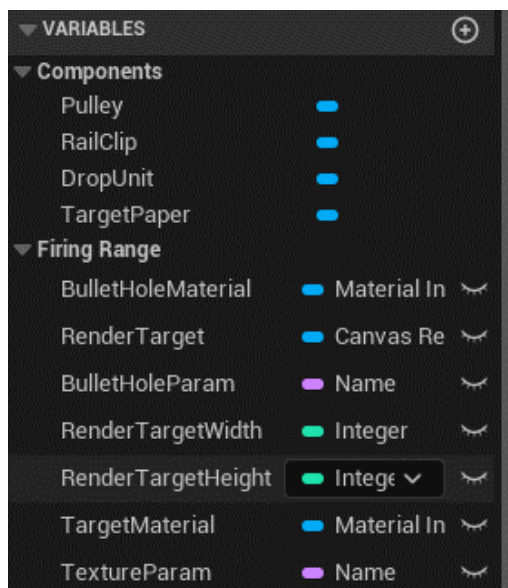
4. Rotate the Rail Clip along the Z Rotation in the Details view – you should see that the Target Paper and Drop Unit now rotate with it. If they weren't children, but instead siblings, as they previously were, the Target Paper and Drop Unit would not Rotate. Put the Z-Rotation back to 0 and then Compile and Save the Blueprint.
5. Select the **Drop Unit**. This provides Collision Support and Sizing for our Target Paper. You cannot see it because of its Material. Change the Material Slot from **Transparent_M** to **WallBricks_M** and you'll be able to see it better. You can reset the Rotation by clicking the Yellow arrow to the right of Rotation.



Hopefully, you can see that this Mesh aligns perfectly with the Target Paper, so we can collide exactly with the underlying Target.

6. Rotate and Position the DropUnit so it is back to exactly covering the Target and reset the material to Transparent_M. (or delete the duplicate and then re-duplicate it).

OK, so we should hopefully understand the Geometry of the Target, next we will look at the Data and Functions.



We have each of the four Components as Variables – each component is always accessible from our Scripts by the same name as specified in the Components View.

We then have 7 Variables. Two of these are Dynamic Materials (both ending in Material), Two are Strings (ending in Param), two are integers (representing a width and height) and the other is a RenderTarget. A Render Target is a special texture that can be drawn onto at run-time.

We use RenderTargets to create things like Mirrors, Computer Screens, Security Cameras, Water Ripples etc. We can draw either a 2D or 3D scene to a RenderTarget and then use it to Texture either a 2D or 3D object. It is easier to understand by viewing it.

See: <https://www.youtube.com/watch?v=a9iho861SIY> for an example.

In our case, it is used to represent the Bullet Holes on the Target. We have two functions, **BeginPlay** and **CustomDrawHole**. Have a look through these yourselves.

BeginPlay sets up our Variables. We create two Dynamic Materials (materials which we can change at run time) – one for the paper and one which is used as our bullet holes.

We then create a 2D RenderTarget of the set width and height (this is the resolution of the target – the higher the resolution, the clearer the holes will be) and then associate the Render Target with the Target Material, so that this target will be using this render target to draw its hole – this way, each target will have its own set of bullet holes.

CustomDrawHole draws the bullet holes onto the render target – the function takes a 2D position, converts it to a 3D position which is then used with the bullet hole material to create a hole at a particular location relative to the RenderTarget scene, we then draw this bullet hole onto the Render Target. The render target is never cleared, so once a bullet hole is added, it stays there – every time we hit the target, it will draw a new hole.

7. Return to the Editor, you can delete the duplicate from your project (make sure you keep the original).
8. Under the World Settings, override the game mode to the **RayCastGameMode**. Drag a couple of instances of the **RailPaperTarget_BP** into the Level, just behind the Tables. The Pulley and Hanger should be visible to help you place them

Run the game and try shooting the Targets – have a look at the holes created and the reaction of the paper.



So, our Target is a Standalone Blueprint that can be interacted with to create bullet holes and collide with.

Step 3: Understanding and Using the TargetFollower

Next, we need to understand how the Follower Works.

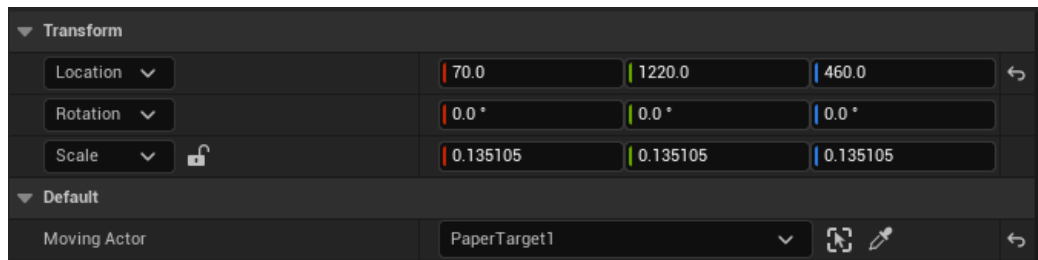
You can remove the Targets you added in the last step.

1. Select **PaperTargetFollower1** in the Editor. In the Details View, can adjust the starting position of the target by sliding the X value of the Location – I have set this up so it appears to slide along the rail, but this is because of the Y and Z position of the Follower.

Change the position using this slider and then **run the game**.

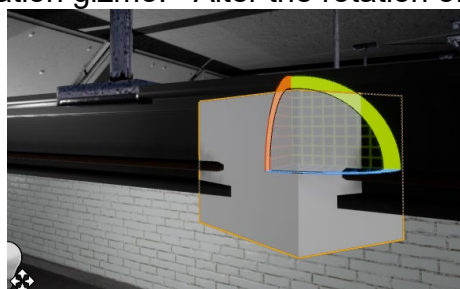


You can see that the Target is positioned directly underneath where the follower is placed. You can also move it via the Transform gizmo – it corresponds to the Red Arrow – make sure you position your camera, so you don't accidentally change the x and z.



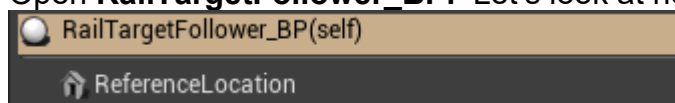
This is because we have associated this follower with an instance of the **RailPaperTarget_BP** called **PaperTarget1**.

2. Try changing the rotation of the Follower.
Select the Follower in the World Outliner and press F (or right-click and select Focus). Press E to switch to the rotation gizmo. Alter the rotation of the cube.



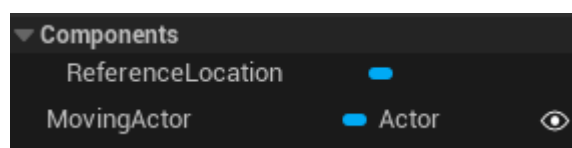
Run the game, and you will see the Rotation has no effect on the Target.

3. Open **RailTargetFollower_BP**. Let's look at how this works.



We have a single Component, which is a StaticMeshComponent, known as **ReferenceLocation** – this is the Root Component, so when we transform the actor (as we did in Steps 1 and 2), it is this component we are changing. The cube is just used as a visible reference to help us see it in game.

You can see that it is not present when we start the game:

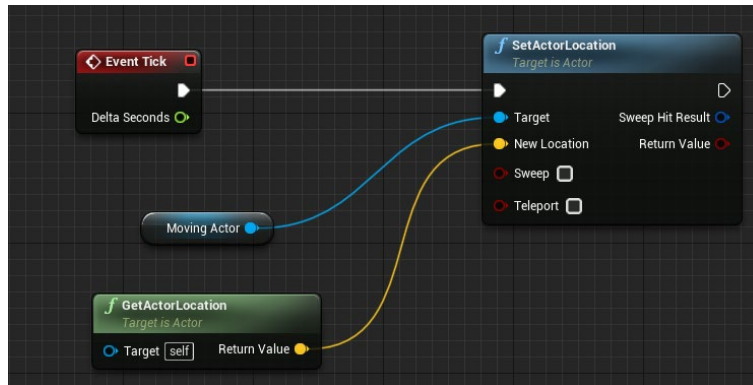


MovingActor is of type: Actor Object Reference, which means that it can be linked with any Actor object, not just our Target class.

This is Polymorphism.

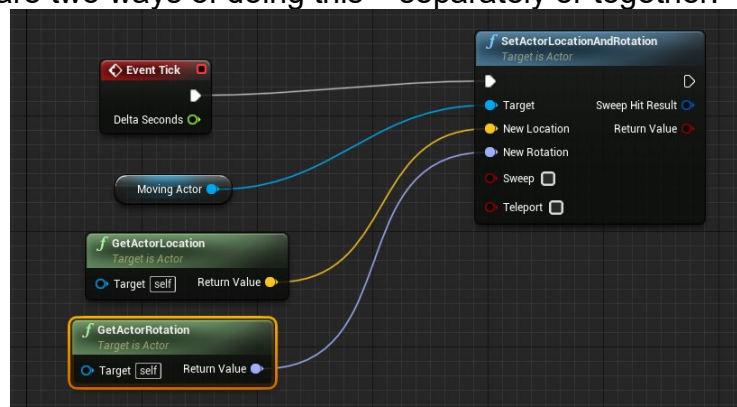
We can store any type of Actor we like; it just happens that we are using it with our Target.

We have one Event, which is Tick, which is UE5's update function –the actor runs this function once per frame. It is very simple:



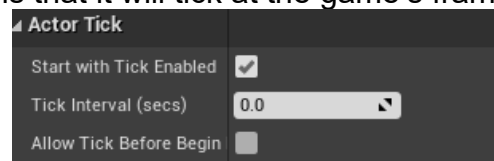
We are simply setting the location of the Actor we are controlling to the same as the Follower's location.

4. Update **Tick** so that it is also setting the Rotation of the Moving Actor to the same as the Followers. There are two ways of doing this – separately or together.

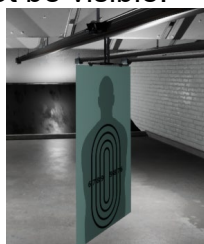


5. Click Class Defaults and then in the Details view, change the **Actor Hidden in Game** property to True. Notice that the Actor is set to Tick Enabled as well, therefore our Tick function is being called – you can disable the tick and control the update rate separately using timed functions.

A Tick Interval of 0.0 means that it will tick at the game's frame rate.



6. Compile and Save the Blueprint. Experiment with the Rotation of the Follower node in the Editor. Notice that the Target is Back-Face Culled – that means, if the rotation is not facing towards the camera, the target will not be visible.

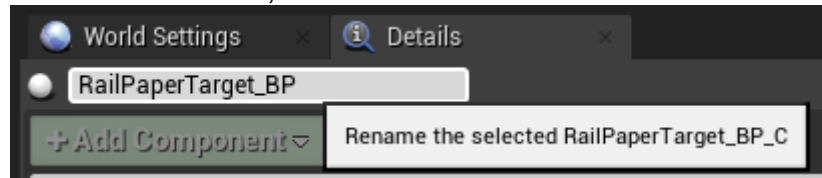


The Cube should have disappeared when we control the game, and you should be able to control the rotation.

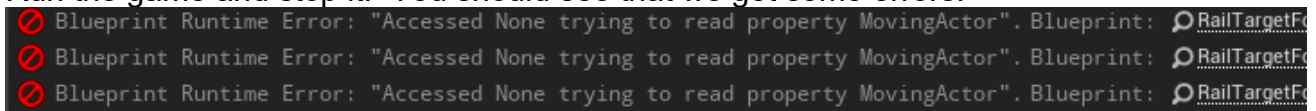
Step 4: Combining the Two Together to create new Followable Targets

Therefore, to add a new Target into the Game, we just need to add an instance of each object, define the position of the follower, and then link it to the Target.

1. Drag in a **RailPaperTarget_BP** into the Level – rename it **NewTarget**. You can do this either in the World Outliner via F2, or in the details view.



2. Drag in a **RailTargetFollower_BP** and rename it **NewFollower**. Position the Follower somewhere in the scene and set its rotation and translation.
3. Right Click in the Outliner and use Create Folder at the top to Create a Folder called **Targets**. Drag our new actors into that Folder.
4. Create another folder called **ControlledTargets** and drag **PaperTarget1**, **PaperTargetControlUnit1** and **PaperTargetFollower1**, and the **RailManager** into that folder.
5. Select New Follower and New Target in the Outliner (you can hold shift to multiple select). Press Ctrl + W to duplicate the two actors. This is a simple way of creating multiple copies – notice it automatically increments the name, so they are unique. Position **NewFollower2**.
6. Run the game and stop it. You should see that we get some errors:



This is because our Tick function is trying to access the MovingActor, but by default it is not set (it is None in UE5 terms – this is the same as **null/nullptr** in C++).

7. Finally, Select NewFollower. In the details view, find the MovingActor field under Default and set this to NewTarget. Do the same with setting NewFollower2 to NewTarget2.

Notice from the Drop down, we can choose any actor we like in the scene – this is because our Object type is Actor – if we changed this to an Object of Type **RailPaperTarget_BP**, it would only show our Targets.

8. Test out our new targets with our gun (try it with both the Projectile Game Mode and the Ray Cast mode). Once you are happy that they are colliding and working, see if you can position the two new Followers so that they are located on any of the four rails that are not controlled by the Control Unit.

Task 3: Managing Placed Actors and Dynamically Spawning Targets

Step 1: Managing our Control Unit Actors

In this task, we will see how we can use the functions of blueprints to store instances and multiple instances of our Target Mechanics blueprints.

1. Locate the **RailManager** object – that should be in our ControlledTargets folder if you completed Task 2 correctly. If it is not there, search the outliner for the word ‘manager’ and find it – if it is not there, drag an instance of the RailManager blueprint into the level and add it to the folder.



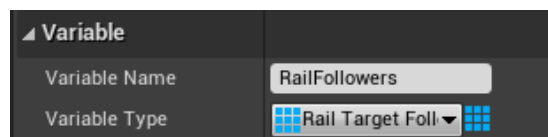
This is an Actor class with no graphical representation (so it doesn't matter where it is in the world) – we want it to oversee storing instances of our Targets, so we can control them from a Script.

2. Click on Edit **RailManager** to open the Blueprint. You can see it just has three variables.

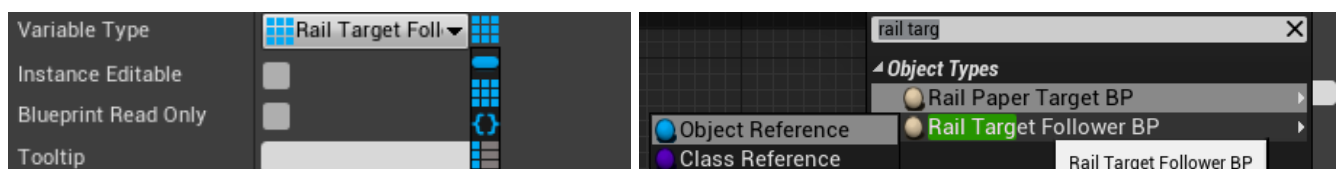


These are all arrays – UE5 has an C++ (STL)-like Array class which is closer to std::vector that we've used in the SFML tutorials – it is a dynamic array that we can add and remove items from at run-time.

3. Look at the details view for each variable, and you will see that the type of the array is our specific blueprints, they store Targets, Followers and Control Units respectively.



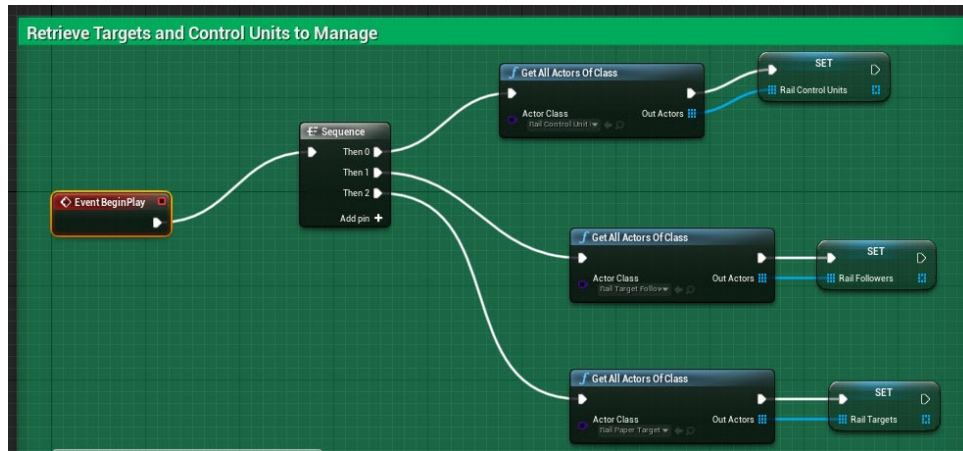
To choose whether we are storing single instances, arrays, or using maps, we use the icon to the right of the variable type.



So, when working with Object Variables, we have two things to do – choose the Type and choose the structure of that type. If you hover over each icon, it will tell you the container type (data structure).

We are populating these structures based on what items are currently in the level. To do this, we have some blueprint nodes that can help us.

4. Locate the Function BeginPlay in the MyBlueprint and open it in the EventGraph.



The **Get All Actors of Class** node is a powerful tool that searches the Game for instances of a particular class and gathers them into an Array. We are doing that for each of our three blueprint types and then storing them in our class members.

5. Right-Click on the Set Rail Targets node in the bottom right and 'Add Breakpoint'. A red marker should be placed on the node. Play the game.

This should cause our game to trigger the breakpoint.

6. Hover over each of the **OutActors** of the three function calls and you should see that it is picking up all the actors that are in our level, including the new targets and followers we have added.

That means we could use these arrays to make changes to each of these elements.

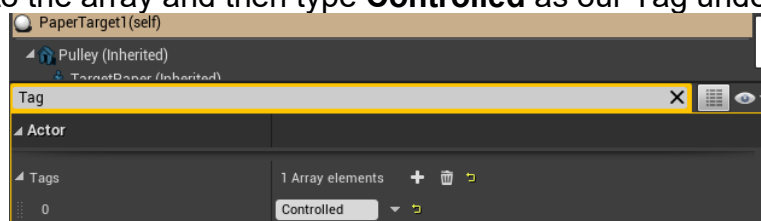
7. Stop the game and then press F9 on our node with the break point to remove it – we can toggle breakpoints on and off easily this way.

OK, so it is picking up all our objects. Let's filter it so it only picks up our Control Unit related objects. Because we haven't got an inheritance hierarchy, we must use a different approach to inheritance and polymorphism to do this. We will use Tags instead.

8. Back in the Editor, select PaperTarget1 in the outliner and then in the Details view of that node, find the **Search Details** box and type **Tag**.

Tags are labels we can associate with Actors. They are just arbitrary strings, which we can use to help classify our Actors.

9. Add an element to the array and then type **Controlled** as our Tag under index 0.



10. Do this for our Control Unit and Follower Actors as well. Notice that search filter should stay active as we switch between our Actors – this is a very useful feature for updating properties of multiple objects.
11. Go to **File -> Save All** once you have done this, as these Tags are saved against the Level itself, so we want to make sure everything is up to date.

Next, we just need to update our Blueprint to make use of these Tags.

12. Go back to our **BeginPlay** function in the Rail manager and break the links between the sequence node and the Get All Actors of Class nodes – you can do this easily using by holding Alt whilst clicking on the link, or pin.
13. Right click in the Event Graph and add a node called **Get All Actors of Class with Tag**. Add three of these nodes into the graph.
14. Add a new variable to the blueprint under the Firing Range Category called ControlTag. The type of this Tag is **Name**. Compile and set the default value to **Controlled** and compile again. You can make this variable **instance editable**.
15. Wire up the blueprint so that it is using the same Actor Class as before in our new nodes, but also has our **ControlTag** variable wired to the Tag pin.

You can delete the **old Get All Actors of Class** nodes once you are done:



Now our arrays will only contain the relevant instances that contain that Tag. Tags can be very useful when we want to group elements of different types or associate actors with a particular gameplay scenario – in this case, actors that are related to the control unit mechanic.

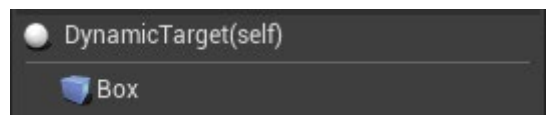
Next, we are going to create a new **Manager** class, which is going to manage our other types of Follower nodes (that aren't using our Control Unit).

Step 2: Making a Dynamically Spawning Target Blueprint

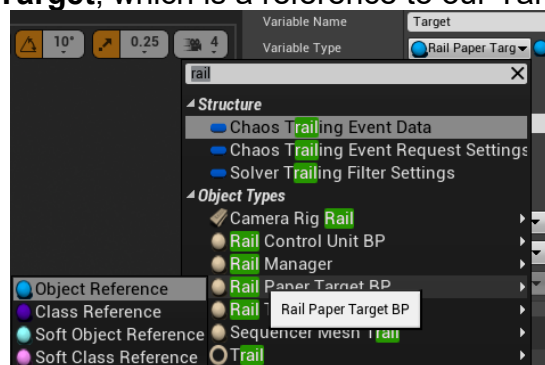
We can see how we can store references to object that have been placed in the level, we also want to be able to manage objects that have been **dynamically added to the level** (spawned).

This time, we will create an actor that can help us manage both the Target and the Follower at the same time and manage that.

1. In the **GameBlueprints** folder, create a new blueprint called **DynamicTarget_BP**. This can extend from Actor.
2. Add a Box Collision Component to it called **Box** and set this as the Root Component.



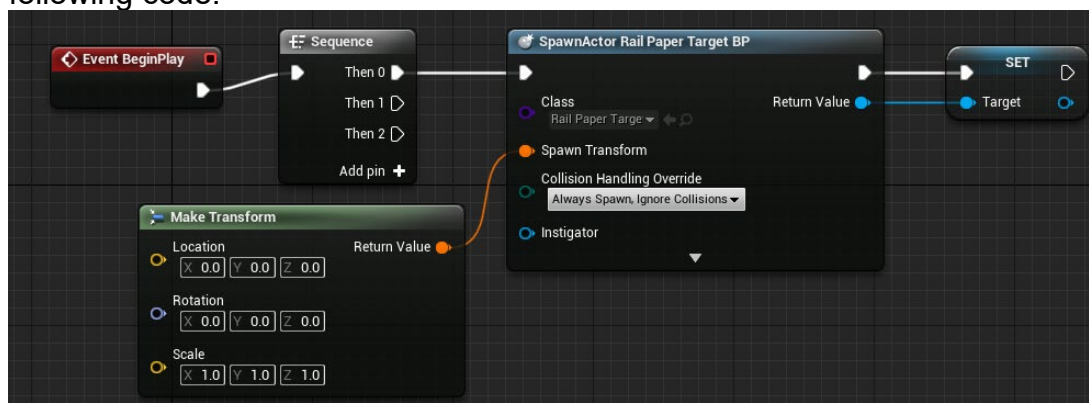
3. Create a variable called **Target**, which is a reference to our Target Blueprint:



4. Create a second variable called **Follower**, which is a reference to our Follower Blueprint. Put both our variables under a Category called **Firing Range**.

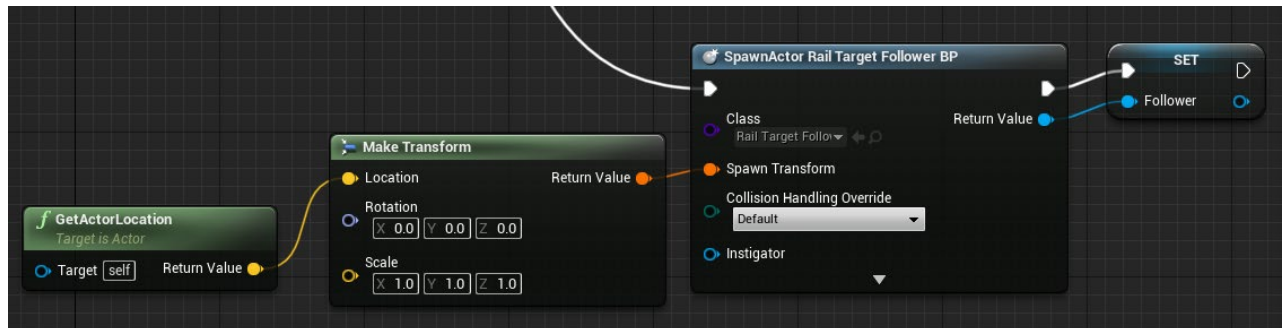
This time we are storing **specific instances** of our Blueprints because we want to directly control these two types of Blueprints. It is highly recommended that these two classes provided in the coursework template are used as the basis for your own sets of Targets – we can use Blueprints like the one we are creating now to handle the movement, creation, and destruction of these Targets.

5. We want this Blueprint to handle the creation of our Target and Follower and then associate them together using scripting, rather than having to place targets in the editor. Therefore, we are going to add this to the BeginPlay event of this Blueprint. Start by adding the following code:



The Spawn node is a SpawnActorFromClass node – it changes name from NONE once we have selected the Class type. Make sure you set the collision handling override – since we know it doesn't matter where this Target is positioned, as it will be following our Follower.

6. Next, to the **Then 1** part of the sequence, add:



This time, we do want to handle collisions on spawning – we are going to spawn the follower, wherever this Blueprint Actor is placed in the world.

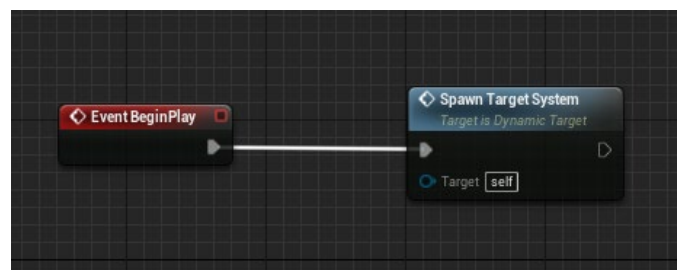
7. Finally, add a **Then 2** node to the sequence and get that to perform the following:



This is programmatically performing the step we did manually – associating the target with the follower.

8. Select all nodes of this event, other than the Begin Play event itself and select **Collapse to Function** from the Right-Click menu. Rename the function **SpawnTargetSystem**.

Now we have a re-usable function which can Spawn our two actors – we are using it with BeginPlay, but we can then re-use it later on if we wanted to (e.g. to get this Actor to respawn targets). You should be left with BeginPlay looking like this:



That's the skeleton of this class created – we can add to it later, such as adding in a delay before spawning, supporting re-spawning, making it active and inactive; changing the spawn position to use the box with a bit of randomness etc., but that's a good enough start for this tutorial.

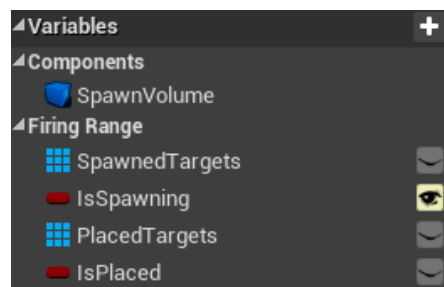
Now we need to create a manager, which can handle multiple Dynamic Targets.

Step 3: Dynamically Spawning Complex Targets

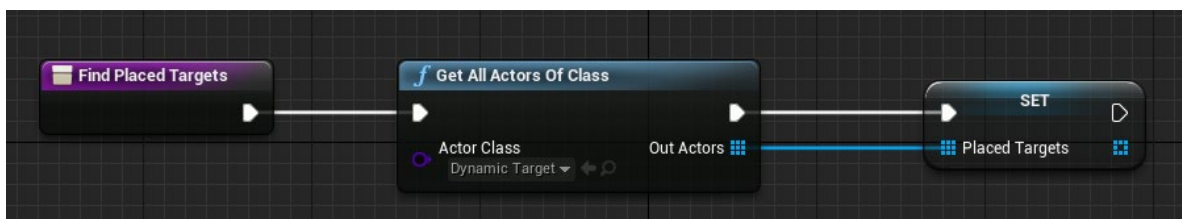
Now we have a Dynamically Spawning Target, we have a few choices – we can either spawn them dynamically; or alternatively, place them in our Level (or even do both!)

We will handle the placed solution first.

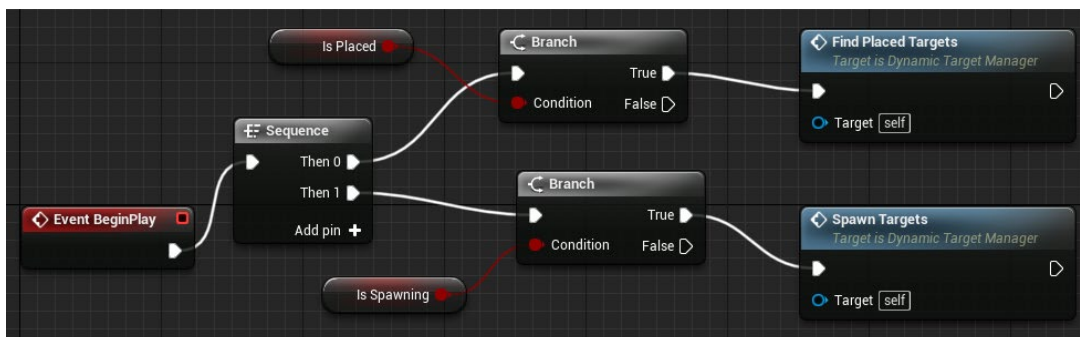
1. Create a new Actor called **DynamicTargetManager**. Add a Box Collision Component to this class as well, called SpawnVolume, and add a variable, which is an array of **DynamicTarget** object references – name this Array, **PlacedTargets**.
2. Create a second array of DynamicTarget object references – name this **SpawnedTargets**.
3. Create a Boolean variable called IsSpawning – make this Instance Editable; make a second Boolean variable called IsPlaced (not instance editable). Add them all to a category called **FiringRange**.



4. Create a Function called **SpawnTargets**. Leave it blank for the moment.
5. Create a Function called **FindPlacedTargets** – add the following code to this function:



6. In BeginPlay, add the following code:

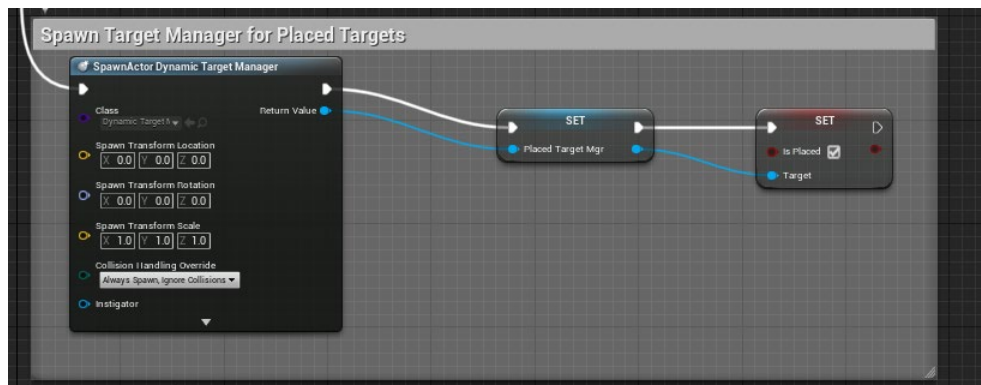


This is another example of using Sequence to make things easier – we want the Spawn Targets branch to run regardless of whether the first branch is true or false – rather than wire up both execution pins, we can use sequence to make this easier.

7. Compile and Save the Blueprint.

This class can now be used in two ways – to Spawn Targets, or control Placed Targets. In the second scenario, we only want one of these classes in Level – we don't want multiple copies of our Placed Actors being referenced. **This is an example of an Actor that is worth adding to the GameMode class (similar to how we have added classes like InputManager to the Game class in our SFML project).**

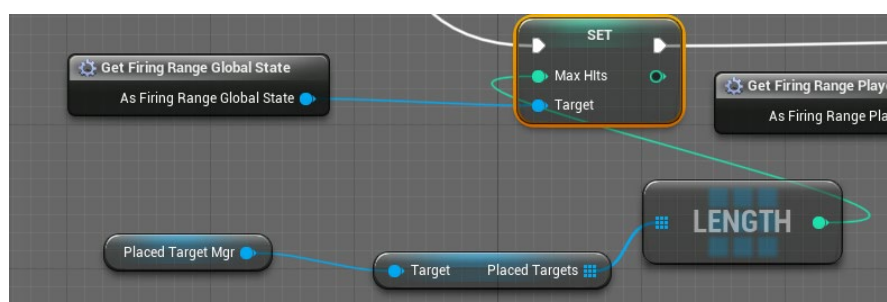
8. Open **FiringRangeGameMode** and add in a class member variable called **PlacedTargetMgr** - this should be an object reference to our **DynamicTargetManager**.
9. Under BeginPlay, add another Sequence Pin and move what is currently being executed from Then 1 so that it starts from Then 2.
10. To **Then 1**, add the following:



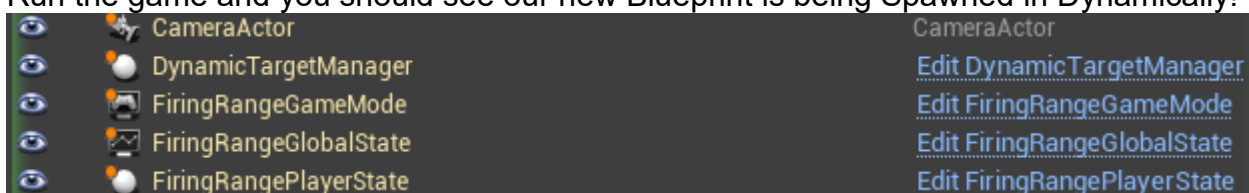
To get the separate Location, Rotation and Scale on the Spawn Actor node, right click on the transform pin and select 'Split Struct Pin'.

11. Next, update the code executed in Then 2, so that the Max Hits is now set from the number of Placed Targets in the Level.

We can do this because BeginPlay of our DynamicTargetManager will be called before this code executes.



12. Compile and Save the Blueprint, and then Set the GameMode Override to None so our FiringRangeGameMode is being used. Run the game and you should see our new Blueprint is being Spawned in Dynamically!



Task 4: Using Inheritance with Polymorphism to create interesting gameplay – moving targets.

In this final task of the tutorial, we will end with something we will be seeing time and time again when using UE5 – by further specializing our classes, we can make our game more interesting, by having a variety of behaviours.

In the case of our RailManager, we are storing arrays of Target Paper, Followers, and the Control Units. **We are not going to inherit from any of these Blueprints since we always want the targets to behave like this**, so they can just be used directly.

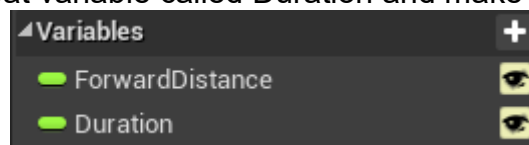
In the case of our DynamicTarget and DynamicTargetManager, however, we do want to use inheritance, and specifically, we want to make use of Polymorphism – this is how we can get each target to behave differently.

We'll implement two examples in this tutorial – **I will leave it up to you to explore creating different hierarchies for your Targets.**

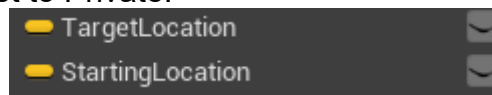
In terms of the movement, the approach used here is just a demonstration – there are better ways of doing this in UE5, which we will also look at in later tutorials.

Step 1: Inheriting from DynamicTarget

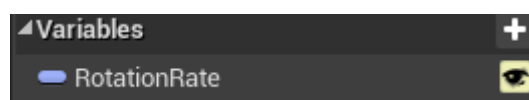
1. Find Dynamic Target and right-click on it – select Create Child Blueprint class to inherit from it. Rename it **MovingTarget**
2. Create a Second child class called **RotatingTarget**
3. Open **MovingTarget** and add a float variable called ForwardDistance, make it Instance Editable. Add another float variable called Duration and make that Instance Editable too.



4. Create a third and fourth variables, called **TargetLocation** and **StartingLocation**. Both should be a Vectors and set to Private.



5. Make the Category of all these variables, **FiringRange** and **Save your progress.**
6. Open **Rotating Target** and create a Rotator Variable called RotationRate, which is Instance Editable. Again, make the category FiringRange.



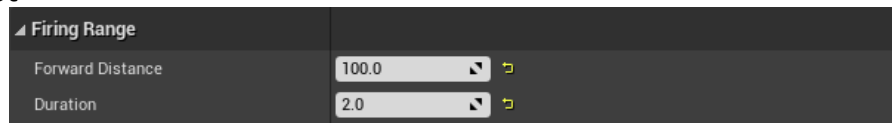
Step 2: Changing the Per Instance Properties

1. Add the two new instances to the Targets folder in the World Outliner – remove the NewTarget and NewFollowers if they are still in there, and any DynamicTarget instances if you added them.



2. You should be able to set the values on our variables on each instance – for example, try setting them to the following:

Moving Target



Rotating Target



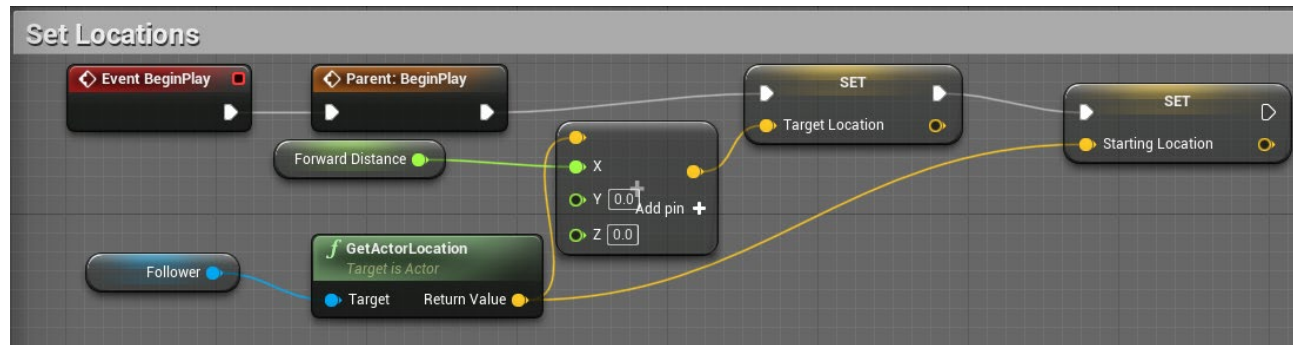
Run the game to make sure it is working as expected. You should have two targets generated, which both should be static.



We need to add the movement logic into each sub-class – we are using inheritance to override / extend behaviour – in this case, there is no existing behaviour in DynamicTarget's Tick function, so we are in essence overriding the behaviour.

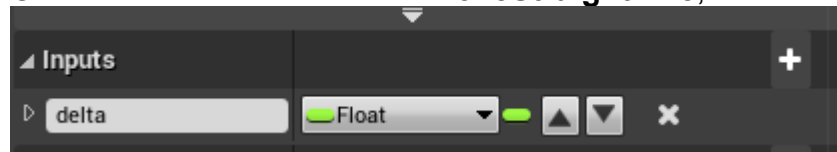
Step 3: Creating the Movement Behaviours – Straight Line Translation to a Target Point.

1. Open **MovingTarget** – we need to start by setting up our Target Location.

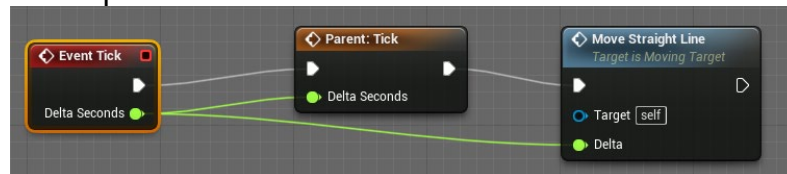


Because we have an inheritance relationship, we call the parent's BeginPlay (DynamicTarget) and then get the location of the Target and add our Forward Distance to it. To do this make a vector + vector node and then split the vector pin to access the X,Y and Z separately. We then set our Target Location.

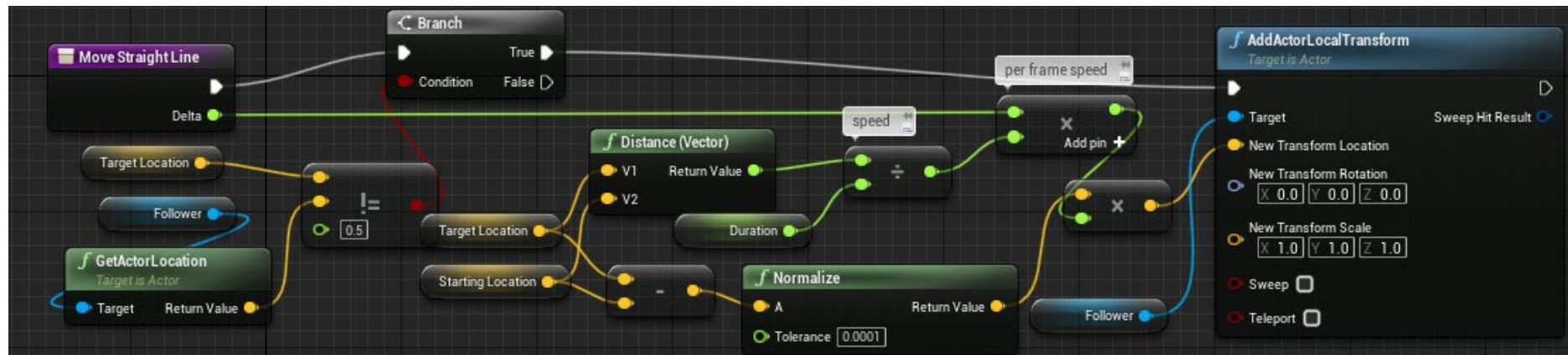
2. Create a New function called **MoveStraightLine**, which takes a float input parameter. Inputs are added via the Details view:



3. Wire up the Tick function to use this function.



4. Complete Move Straight Line with the following – this is an example where Blueprints becomes harder to interpret than the equivalent C++ code – Mathematical Linear Algebra.

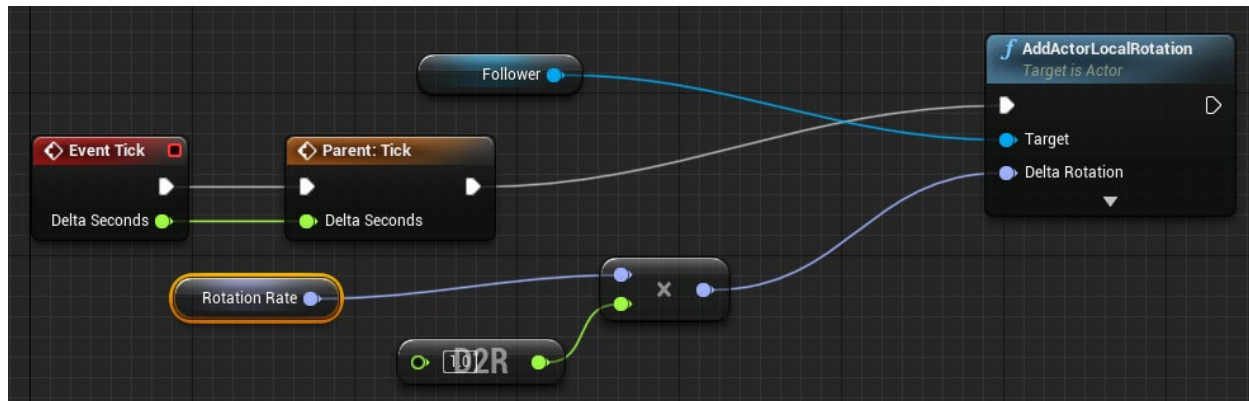


What this code is doing is :

- Calculating the vector between the start and target location using vector subtraction. This vector is normalized to give a direction to the target.
 - The speed is calculated via $\text{distance} / \text{time}$, so we measure the distance between the two and calculate the speed of movement needed to reach the target within the specified Duration.
 - This speed is in metres / second, so needs to be multiplied by the frame time to give us the speed in metres/frame.
 - The Direction is multiplied by this speed, to give us a Velocity or Displacement vector – if we add this vector each frame to the current position of the follower, we reach the destination.
 - The Branch at the start says that this code will only run if the Target Location is not the same as the current location – if it is, we have already reached our destination, so stop moving. Because of Floating pointer error, UE5 has a tolerance value to its checks – in this case the tolerance is 0.5 – is we are within 0.5 units of the Target, treat it that we reached the target.
- 5. Compile and Save the Blueprint and run the code.**

Step 4: Creating the Movement Behaviours – Rotation around a Point

1. Open **RotatingTarget**. There is no setup code here as our RotationRate variable can be used directly in our Tick code.
2. Complete Event Tick as follows:



This is much simpler than our Translation code – we are just going to continuously rotate our Target by Adding the RotationRate to the current Rotation of our Target Follower. The only thing we need to be aware of is that a Rotator is specified in Radians, we are using Degrees, so we need to convert Degrees to radians – we use a function called Degrees to Radians to convert 1 degree to a radian and then Scale the Rotation Rate by to get the Rotator in Radians to then use with the AddActorLocalRotation node.

3. Compile and Save the Blueprint and Test the code. If the Rotation is not working, make sure you have wired up the Rotation code for the Follower as covered earlier in the tutorial (Step 3 of Task 2).

Additional Task: Creating Child Classes

Step 5: Creating your own types of Targets and Experimenting with Rotating/MovingTarget

So, in this tutorial, we have now got a system for creating Targets that spawn a Paper Target and a Follower, which can control its rotation and child blueprints that can then procedurally animate the Targets by overriding the Tick function – we have a Manager class which can either detect these placed Targets or spawn them themselves – this has been integrated into the game mode, alongside a custom Player State and a custom Global Game State.

In our next tutorial using the Template, we will progress these steps, so it is advised that you save your project and Zip it up as **SEG_Tutorial1.zip** and store it somewhere safe, as we will be using it in our next Unreal Tutorial.

For now, the following tasks are suggested:

- Experiment with Placing Rotating and Moving Targets and experimenting with those Instance values – set the distance and duration on each moving target (try negative distances) and set the Rotation Rate on the Rotating targets (try different axes and speeds).
- Create your own child classes of DynamicTarget.
- Try and plan out your series of target types that you want to incorporate and the types of movement you wish to see in your own implementation.

THAT CONCLUDES TODAY'S TUTORIAL